Static application security testing (SAST)

1. Introduction

Static Application Security Testing (SAST) is a security testing approach that analyzes an application's source code, bytecode, or binaries without executing the program. The primary objective of SAST is to identify security vulnerabilities, insecure coding practices, and potential weaknesses early in the software development lifecycle.
This report presents the results of a Static Application Security Testing (SAST) assessment performed on the E-commerce application using SonarCloud. The analysis focuses on identifying security vulnerabilities and security hotspots within the application's source code.

2. Objective of SAST

The objectives of this Static Application Security Testing assessment are to:

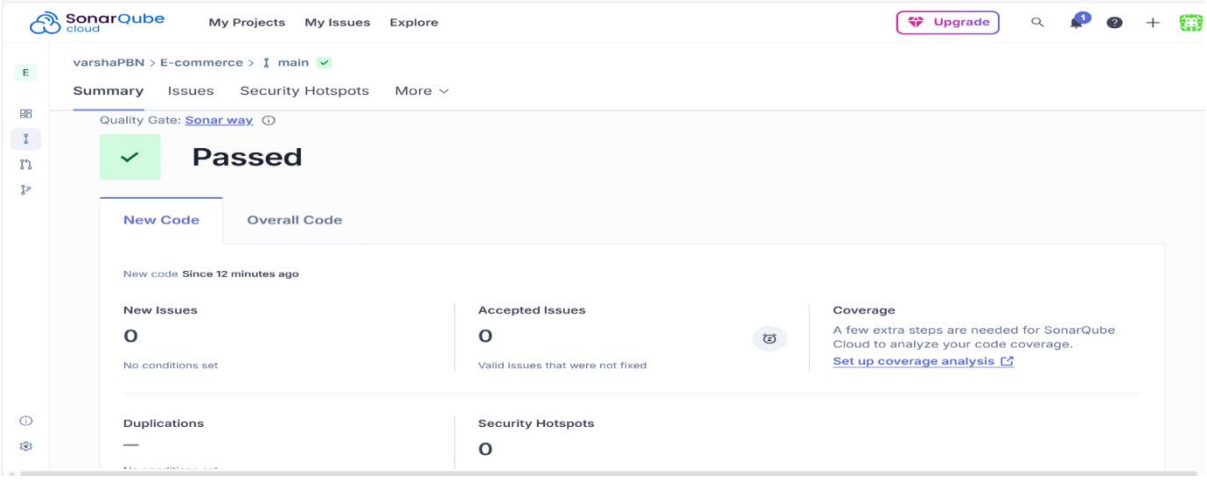
- Identify security vulnerabilities in the source code
- Detect insecure coding patterns
- Highlight security-sensitive areas requiring manual review
- Improve the overall security posture of the application
- Support secure software development practices

3. Tools Used

- Tool Name: SonarCloud
- Testing Type: Static Application Security Testing (SAST)
- Platform: Cloud-based
- Integration**:** GitHub
- Analysis Mode: Static source code analysis (no runtime execution)

4. Scope of Assessment

Application Name : E-commerce
SonarCloud Organization : varshaPBN
Branch Analyzed : main
Scan Type : Static Application Security Testing (SAST)
Tool Used : SonarCloud
Scan Status : Completed
Quality Gate : Sonar way (Passed)



5. Technology Stack and Codebase Overview

Component	Lines of Code	Security Issues	Reliability Issues	Maintainability Issues	Security Hotspot
Backend	1762	28	2	39	7
Frontend	11,215	0	121	277	1
Total	12,977	28	123	316	8

6. Methodology

The Static Application Security Testing assessment was conducted using the following methodology:

1. Source code was hosted in a GitHub repository.
2. SonarCloud was integrated with the repository using GitHub authentication.
3. The repository was analyzed automatically by SonarCloud.
4. Static analysis was performed without executing the application.
5. Results were reviewed through the SonarCloud dashboard.

7. Summary of Security Findings

7.1 Vulnerabilities by Severity

Severity	Count
Blocker	28
Critical	0
Major	0
Minor	0
Info	0

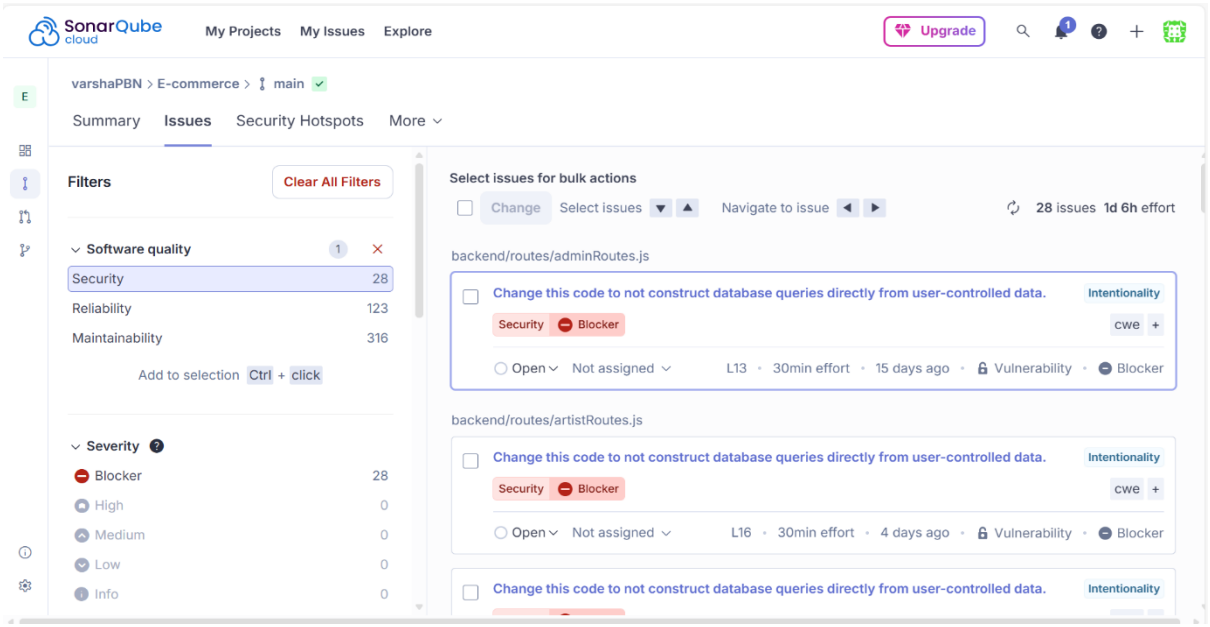
7.2 Security Hotspots

Category	Count
Total Hotspots	8
Reviewed	0
To Review	8
Review Completion	0%

8. Detailed vulnerability findings

Example Vulnerability: Injection Risk

- Severity: Low / Medium (as identified in scan)
- Type: Vulnerability
- Affected Area: Backend routes
- Description:
  User-controlled input is used in backend operations without sufficient validation, which may lead to injection-related security risks.
- Security Risk:
  Unauthorized data access or manipulation.
- Recommendation:
  Validate and sanitize all user input and use secure query handling mechanisms such as parameterized queries or ORM frameworks



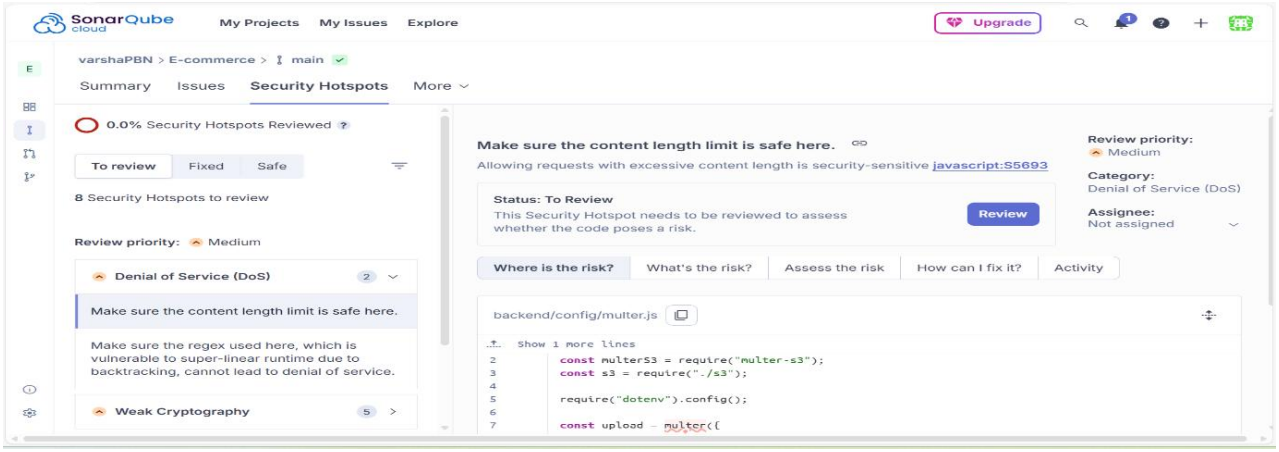
9. Security Hotspot

Security hotspots represent sensitive areas of code that require manual security review.

Example Security Hotspot

- Category: Denial of Service (DoS)
- Description:
  Lack of proper input constraints may allow excessive resource consumption.
- Status: To Review

- Recommendation:
  Apply request size limits and input validation to prevent potential misuse.



10. Risk Assessment

Likelihood: Medium
Impact: Medium
Overall Risk Rating: Medium

The presence of multiple vulnerabilities and unreviewed security hotspots indicates the need for remediation before production deployment.

11. Limitations of SAST

- SAST does not identify runtime vulnerabilities
- False positives may exist
- Manual review is required for security hotspots
- Dynamic testing and penetration testing are not included

12. Conclusion

The Static Application Security Testing (SAST) assessment using SonarCloud identified several security vulnerabilities and security hotspots within the E-commerce application. While no high or critical severity issues were observed, addressing the identified low and medium severity findings will improve the application's overall security posture. It is recommended to remediate the issues, review all security hotspots, and perform a re-scan to validate fixes.